

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

1. Giới thiệu	4
1.1. Kiến trúc tổng quan: Monolith → SOA → Microservices	4
1.2. Vị trí của sách	4
1.3. Tổng quan 12 chương	4
1.3.1. Phần I: Nền tảng (Ch.1-3)	4
1.3.2. Phần II: Giao tiếp & Dữ liệu (Ch.4-7)	5
1.3.3. Phần III: Hạ tầng & Vận hành (Ch.8-12)	5
1.4. Lộ trình đọc theo nhu cầu	5

1.

CHƯƠNG 1.

Giới thiệu

1.1. Kiến trúc tổng quan: Monolith → SOA → Microservices

Kiến trúc phần mềm đã trải qua một quá trình tiến hóa kéo dài hơn hai thập kỷ:

Monolith (trước 2010) — Toàn bộ ứng dụng nằm trong một codebase và được triển khai thành một artifact duy nhất. Đơn giản, hiệu quả cho nhóm phát triển quy mô nhỏ, nhưng gặp giới hạn khi hệ thống lớn dần: triển khai chậm, khó mở rộng, bị khóa chặt vào một công nghệ (technology lock-in).

SOA (2000-2015) — Service-Oriented Architecture tách hệ thống thành các service giao tiếp qua Enterprise Service Bus (ESB). SOA giải quyết bài toán tích hợp giữa các hệ thống lớn, nhưng chính ESB lại trở thành điểm nghẽn (bottleneck) và điểm lỗi duy nhất (single point of failure).

Microservices (2014-nay) — Tiến hóa từ SOA với nguyên tắc “smart endpoints, dumb pipes”: các service nhỏ, tự trị, triển khai độc lập. Phổ biến nhờ Netflix, Amazon, Uber chứng minh hiệu quả ở quy mô lớn.

1.2. Vị trí của sách

Cuốn sách này **không** hướng dẫn xây dựng microservices từ con số không. Thay vào đó, nó giúp người đọc **hiểu** microservices: tại sao cần, khi nào dùng, trade-off là gì, và chuyển đổi từ monolith như thế nào. Sử dụng KBM, một hệ thống LMS thực tế, làm case study xuyên suốt, mỗi chương phân tích: *bài toán thực tế* → *pattern giải quyết* → *áp dụng cho KBM* → *lộ trình chuyển đổi*.

1.3. Tổng quan 12 chương

1.3.1. Phần I: Nền tảng (Ch.1-3)

Ch.1 — Tổng quan SOA & Microservices — từ lý thuyết đến thực tế (Netflix, Amazon, Uber)

Ch.2 — Domain-Driven Design — Bounded Contexts, Context Map, Event Storming, và tổ chức đội phát triển

Ch.3 – Thiết kế API – REST, quản lý phiên bản (versioning), tiến hóa schema, tài liệu hóa API

1.3.2. Phần II: Giao tiếp & Dữ liệu (Ch.4-7)

Ch.4 – Giao tiếp đồng bộ – REST, gRPC, OpenFeign, và Resilience Patterns

Ch.5 – Giao tiếp bất đồng bộ – Kafka, kiến trúc hướng sự kiện (event-driven), message broker

Ch.6 – Saga Pattern – giao dịch phân tán, orchestration và choreography

Ch.7 – Quản lý dữ liệu – database-per-service, CQRS, Event Sourcing

1.3.3. Phần III: Hạ tầng & Vận hành (Ch.8-12)

Ch.8 – API Gateway – routing, cross-cutting concerns, Spring Cloud Gateway

Ch.9 – Bảo mật – JWT, OAuth2, RBAC

Ch.10 – Chuyển đổi thực tế – Strangler Fig, tách cơ sở dữ liệu, lộ trình chuyển đổi

Ch.11 – Observability – log, metrics, tracing, cảnh báo

Ch.12 – Triển khai & DevOps – Docker, CI/CD, chiến lược triển khai, IaC

Lưu ý về kiểm thử (Testing): Cuốn sách không thiết kế một chương kiểm thử độc lập. Thay vào đó, để tiếp cận từ góc nhìn của nhà phát triển và thiết kế hệ thống, các chiến lược kiểm thử như kiểm thử hợp đồng (contract testing), kiểm thử thành phần (component testing) và Chaos Engineering được lồng ghép trực tiếp vào các chương chuyên đề tương ứng (Chương 3, 10, 11) và hệ thống bài tập thực hành.

1.4. Lộ trình đọc theo nhu cầu

Sách được thiết kế để đọc tuần tự, nhưng tùy mục tiêu, người đọc có thể chọn một lộ trình phù hợp:

Nhóm độc giả	Gợi ý lộ trình
Sinh viên năm 3-4 (lần đầu tiếp cận)	Đọc tuần tự Ch.1 → Ch.12, nhưng đọc lướt các mục đánh dấu “nâng cao” ở Ch.1-2 và quay lại sau. Sử dụng Phụ lục A (Bảng thuật ngữ) khi gặp từ lạ. Ưu tiên hiểu phần “Vấn đề” đầu mỗi chương trước khi đi vào pattern.
Lập trình viên junior/mid đang bảo trì monolith	Bắt đầu Ch.1 (khi nào nên/không nên) → Ch.2 (Bounded Context) → Ch.10 (Strangler Fig, lộ trình chuyển đổi), rồi quay lại Phần II khi cần kỹ thuật cụ thể.
Đang chuẩn bị phỏng vấn hệ thống/kiến trúc	Đọc kỹ Phần II (Ch.4-7) + Ch.9, sau đó luyện toàn bộ phần Bài tập (Trade-off, ADR, Debugging) ở cuối sách.
Trưởng nhóm/quyết định kiến trúc	Ch.1 (Khung ra quyết định) → Ch.2 (Conway’s Law) → Ch.10 → Kết luận (bảng heuristic “khi nào dùng pattern nào”). Phần lý thuyết chi tiết tra cứu khi cần.